Préparation de l'environnement :
Installer **Poetry pour gérer les dépendances du projet (semblable à MAVEN)**

1. **Sur MAC - Mise à jour (Update) des certificats SSL de Python 3.14**
   ```
   open "/Applications/Python 3.14/Install Certificates.command"
   ```

2. **Installer poetry**
   ```
   curl -sSL https://install.python-poetry.org | python3 -
   ```
   ou
   ```
   python3.13 -m pip install poetry==2.5.1
   ```

3. **Ajouter Poetry au PATH pour pouvoir utiliser la commande dans le Terminal**
   ```
   echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc source
   ~/.zshrc
   ```

4. **Vérifier la version de Poetry**
   ```
   poetry --version
   ```

WINDOWS
https://gist.github.com/Isfhan/b8b104c8095d8475eb377230300de9b0

Tailwind CSS est un framework CSS qui permet de créer et personnaliser rapidement l'apparence d'une interface web : couleurs, boutons, espacements, tailles et adaptation aux mobiles.

5. **Installer TailwindCSS avec node.js**
   ```
   npm install -D tailwindcss@4
   ```
6. **Télécharger et Installer TailwindCSS sans node.js**
   ```
   mkdir -p "$HOME/.local/bin"

   case "$(uname -m)" in
     arm64) TAILWIND_ARCH="arm64" ;;
     x86_64) TAILWIND_ARCH="x64" ;;
   esac

   curl -fL
   "https://github.com/tailwindlabs/tailwindcss/releases/latest/do
   wnload/tailwindcss-macos-${TAILWIND_ARCH}" \
     -o "$HOME/.local/bin/tailwindcss"
   ```

7. **Donne au fichier Tailwind la** permission d'être exécuté **comme un programme. À faire une seule fois après le téléchargement.**
   ```
   chmod +x "$HOME/.local/bin/tailwindcss"
   ```

8. **Lancer Tailwind et affiche** l'aide avec les options disponibles**. Cela permet aussi de vérifier qu'il fonctionne.**
   ```
   tailwindcss -help
   ```

9. **Vérifier la version**
   ```
   Tailwindcss -v
   ```
10. **Accéder au bin**
    ```
    cd $HOME/.local/bin/
    ```
11. **Visualiser l'aide de cet executable:**

```
./tailwindcss
```

12. Pour qu'il soit accessible sans entrer dans ce dossier je le déplace au chemin :

```
sudo mv tailwindcss /usr/local/bin
```

13. Créer et accéder au dossier du projet :

```
mkdir PROJECT-10
cd PROJECT-10
```

14. Initialiser le projet

```
poetry init
```

15. Valider sans écrire

```
Author = Utilisateur
Would you like to define your main dependencies interactively?
(yes/no) [yes] no
Would you like to define your development dependencies
interactively? (yes/no) [yes] no
..
Générer le projet : YES
```

16. Visualiser le contenu du dossier au Terminal :

```
ls
pyproject.toml
```

Un seul fichier est créé

17. Visualiser le contenu du fichier au Terminal :

```
cat pyproject.toml
```

18. Installer la dépendance de flask

On peut specifier la version sinon il installe la dernière

```
poetry add flask==2.3.3
```

19. Installer la dépendance de openai

On peut specifier la version sinon il installe la dernière

```
poetry add openai==0.27.10
poetry add openai@latest
```

20. Installer la dépendance pour lire le fichier

```
poetry add python-dotenv==1.0.0
```

21. Ouvrir le PROJECT-10 dans PyCharm

On trouve 2 fichiers créés

22. Spécifier l'interpréteur Poetry

- **PyCharm/settings**
- **Chercher : interpreter**
- **Cliquer sur : Add interpreter**
- **Choisir la version de Poetry que nous venons d'installer**

NB : Pour avoir le chemin de poetry :

```
which poetry
```

23. Créer le fichier : app.py

```python
from flask import Flask

#créer l'application web Flask et la stocke dans la variable app
#__name__ : indique à Flask le module courant, ce qui l'aide à
localiser notamment les dossiers templates et static.
```

```
app = Flask(__name__)

#on met un route en mentionnant la racine comme page d'accueil
@app.route('/')
#on crée une fonction, dans un premier temps -> elle affiche juste
un message
def home():
    return 'Hello World!'

#démarrer le serveur lorsque tu lances directement le fichier
if __name__ == '__main__':
    #cette ligne permet de lancer ce fichier et de lancer le
serveur Flask
    app.run(debug=True)
    #possibilité de spécifier le port voulu

    #app.run(debug=True, host='0.0.0.0', port=5000)
```

Exécuter : app.py

Soit au terminal :

```
python app.py
```

Soit au terminal :

```
flask --app app run
```

Soit au terminal : puisque le nom de l'app est : app - NB : mode debug est off ici

```
flask run
```

Soit au terminal : puisque le nom de l'app est : app - NB : mode debug est on ici

```
flask run -debug
```

Soit au terminal : sans activer l'environnement

```
poetry run python app.py
```

Soit au terminal : activer l'environnement

Nécessite d'ajouter un pluggin

```
poetry self add poetry-plugin-shell
poetry shell
python app.py
```

Soit en cliquant sur le bouton : exécuter

24. Ouvrir dans le navigateur :

http://127.0.0.1:5000/

25. Créer un dossier et un fichier : templates/index.html

26. Écrire dans : index.html

Html5

27. Le contenu de index.html

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <title>Document</title>
</head>
```

```
<body>
 <h1>Lorem </h1>
</body>
</html>
```

28. Dans app.py on importe :

```
from flask import Flask, render_template
```

29. Changer :

```
def home():
    return render_template('index.html')
```

30. Executer et lancer

http://127.0.0.1:5000/

31. Créer dossier :

```
mkdir -p static/css
```

32. Dans static/css ajouter le fichier : input.css il contient

```
@import "tailwindcss";
```

33. Lancer la génération du CSS

```
tailwindcss -i ./static/css/input.css -o ./static/css/output.css -watch
```

NB: Laisse ce Terminal ouvert pour actualiser automatiquement le CSS lorsque tu modifies tes pages.

34. Ajouter dans index.html

```
<head>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/output.css') }}">
```

Partie	Rôle
`<link>`	Relie la page à une ressource externe.
`rel="stylesheet"`	Précise que la ressource est une feuille de style CSS.
`href="..."`	Indique l'adresse du fichier à charger.
`{{ ... }}`	Demande au moteur de templates Jinja d'évaluer l'expression et d'insérer son résultat dans le HTML.
`url_for('static', filename='css/output.css')`	Demande à Flask de construire l'URL du fichier `css/output.css` situé dans le dossier `static`.

Maintenant on peut utiliser les classes de tailwindcss

35. Ajouter la classe de tailwindcss dans index.html

```
<h1 class="text-3xl font-bold text-blue-600">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Accusantium animi cumque delectus dicta earum excepturi, hic iste maxime, molestiae neque non perspiciatis placeat porro quae sit soluta unde velit voluptatum. </h1>
```

36. Avant on devait ajouter dans input.css:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

37. Maintenant on doit juste ajouter dans input.css:

```
@import "tailwindcss";
```

38. Ajouter dans input.css: Si Tailwind ne détecte pas automatiquement les fichiers HTML

```
@source "../../templates";
```

39. Cette commande transforme ton fichier source Tailwind en un fichier CSS utilisable par le navigateur, puis le met à jour automatiquement :

```
tailwindcss -i ./static/css/input.css -o
./static/dist/css/style.css –watch
```

Cette ligne crée un dossier dist

40. Changer dans index.html

```
<link rel="stylesheet" href="{{ url_for('static',
filename='css/output.css') }}">
```

Élément	Rôle
`tailwindcss`	Lance le compilateur Tailwind.
`-i ./static/css/input.css`	Indique le fichier CSS d'entrée contenant `@import "tailwindcss";`.
`-o ./static/dist/css/style.css`	Indique où écrire le CSS généré.
`--watch`	Surveille les changements et régénère le CSS automatiquement.

NB : Laisse cette commande tourner dans un Terminal et lance Flask dans un autre.

Pour arrêter Tailwind : **Ctrl+C**.

41. Revoir l'exécution

Suppression d'une donnée de class CSS par exp : text-blue-600

Tu verras qu'elle est créé automatiquement